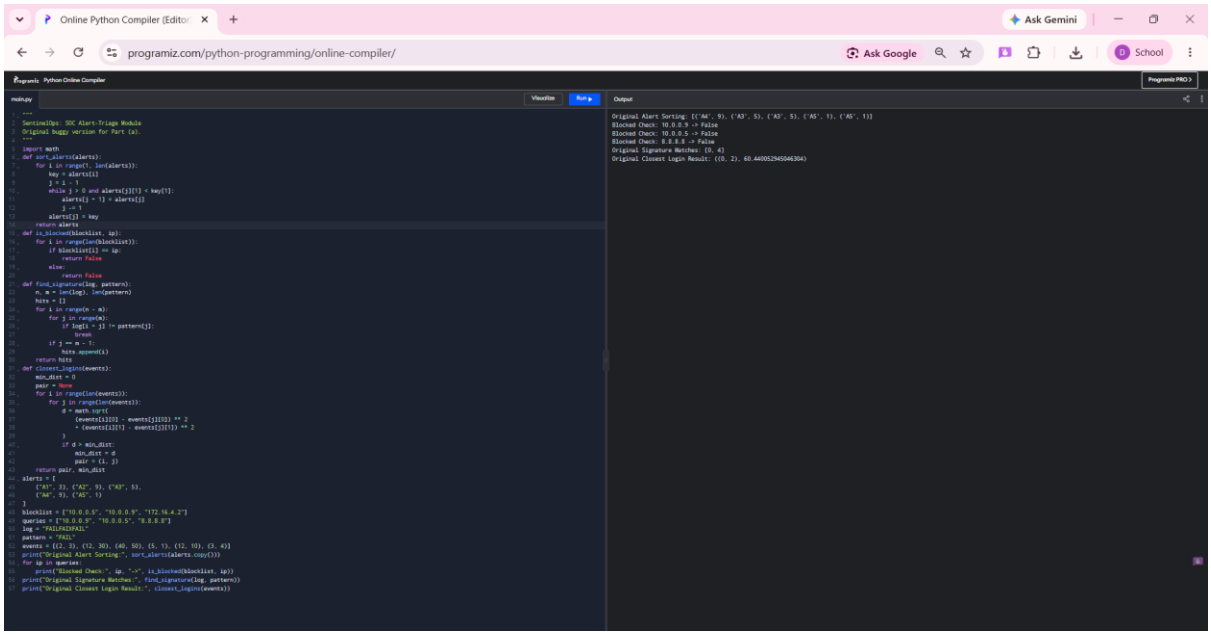
CO2 – AT 2

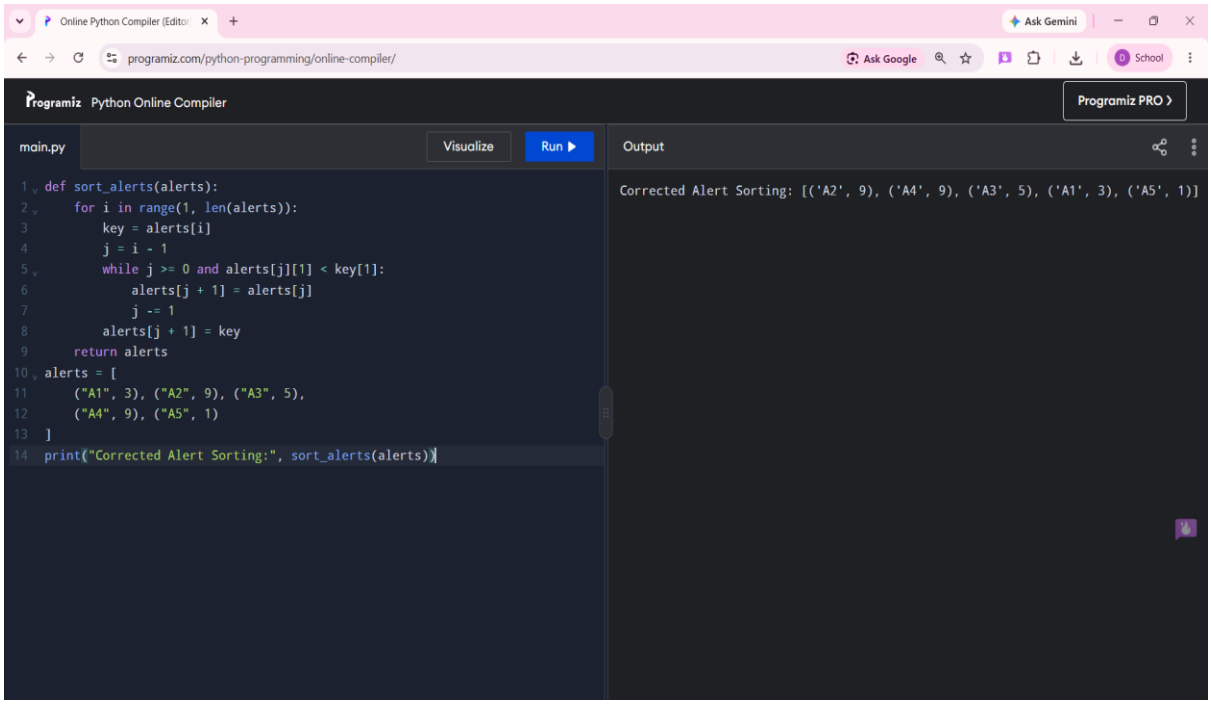
IMPLEMENTATION SCREENSHOTS

PART (a): REPRODUCE AND IDENTIFY THE DEFECTS

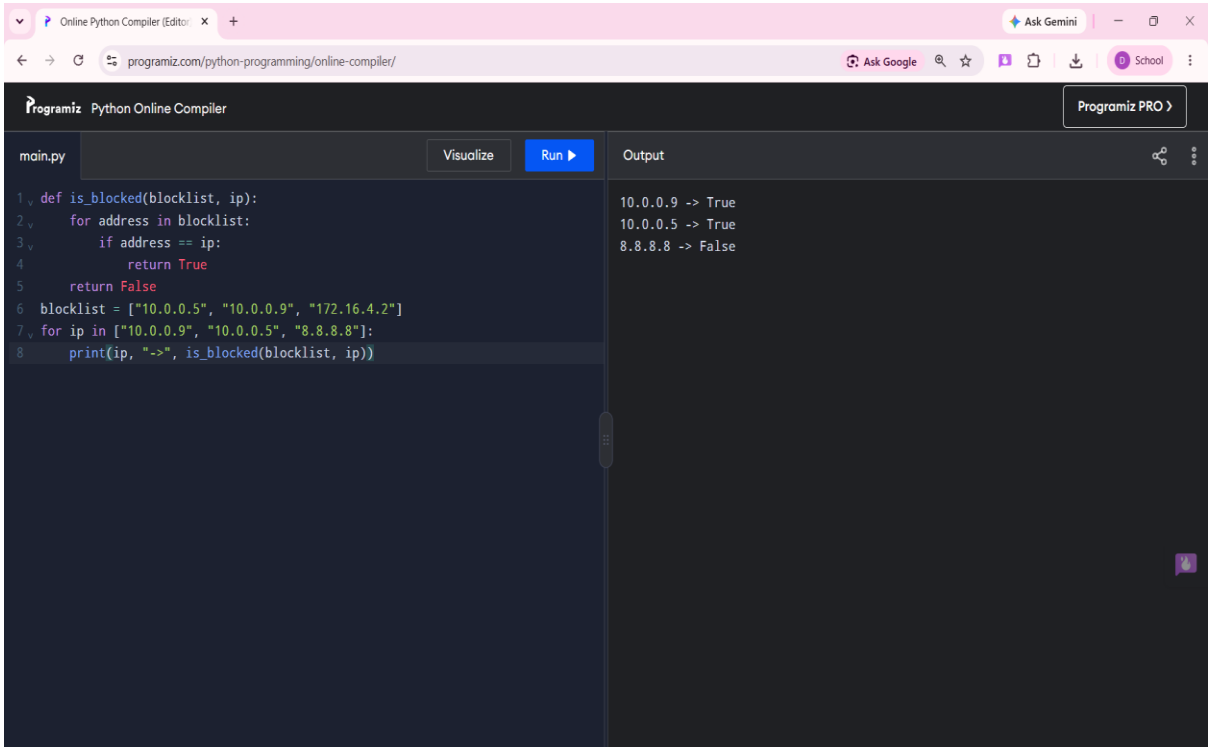


PART (b): EXPLAIN THE ROOT CAUSES AND DEBUG SYSTEMATICALLY (12 MARKS)

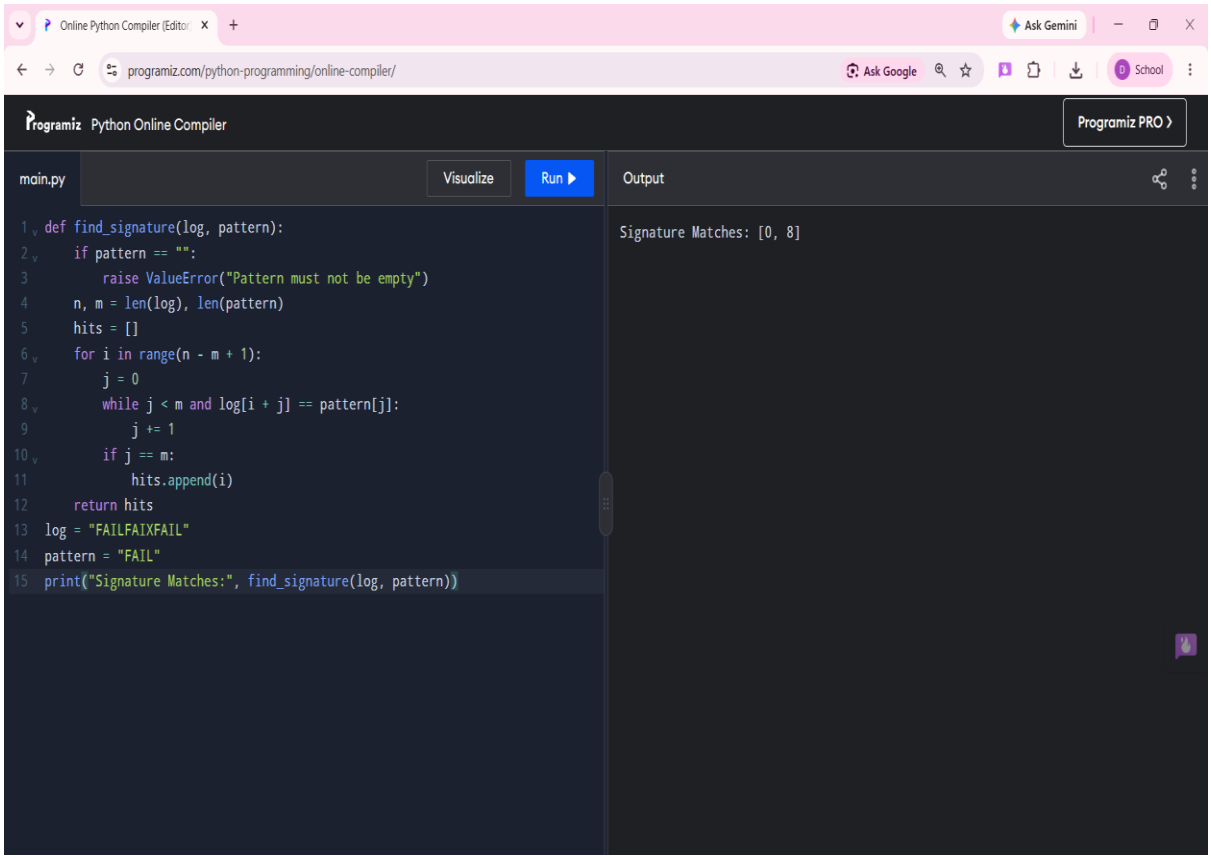
Debugging sort_alerts()



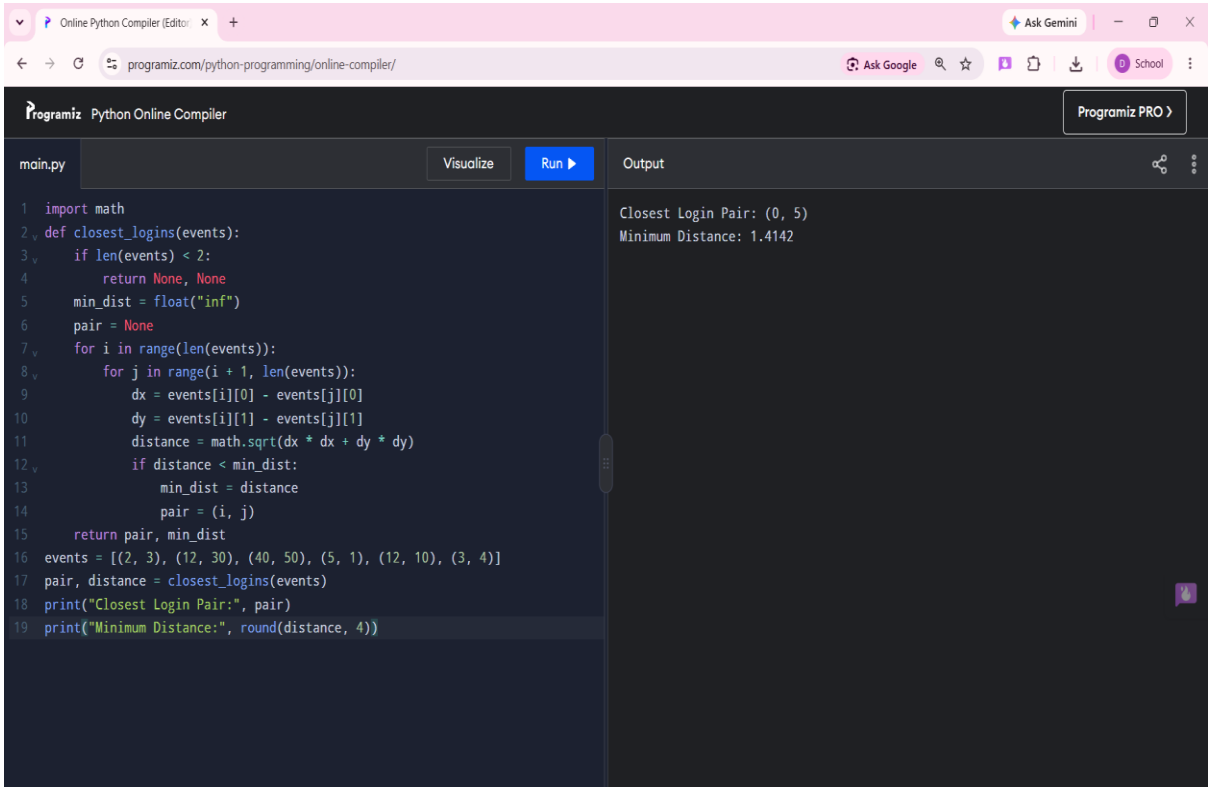
Debugging is_blocked()



Debugging find_signature()

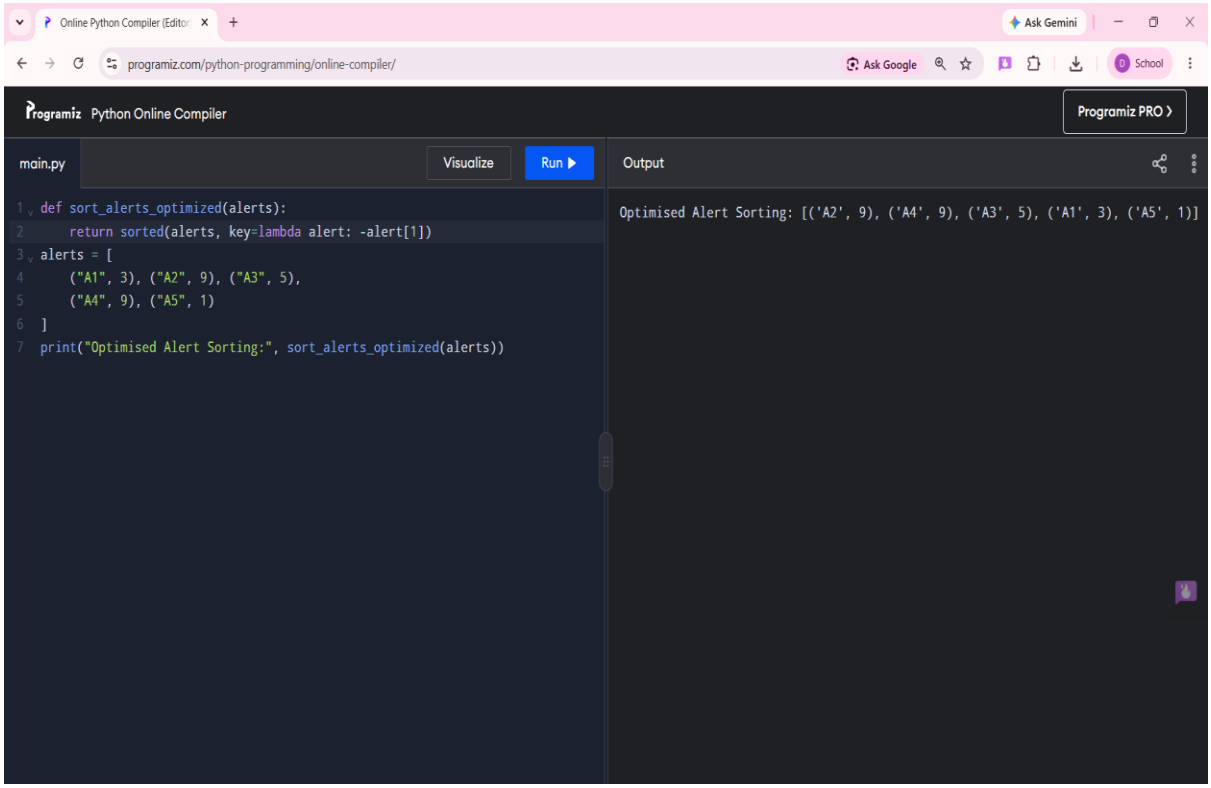


Debugging closest_logins()

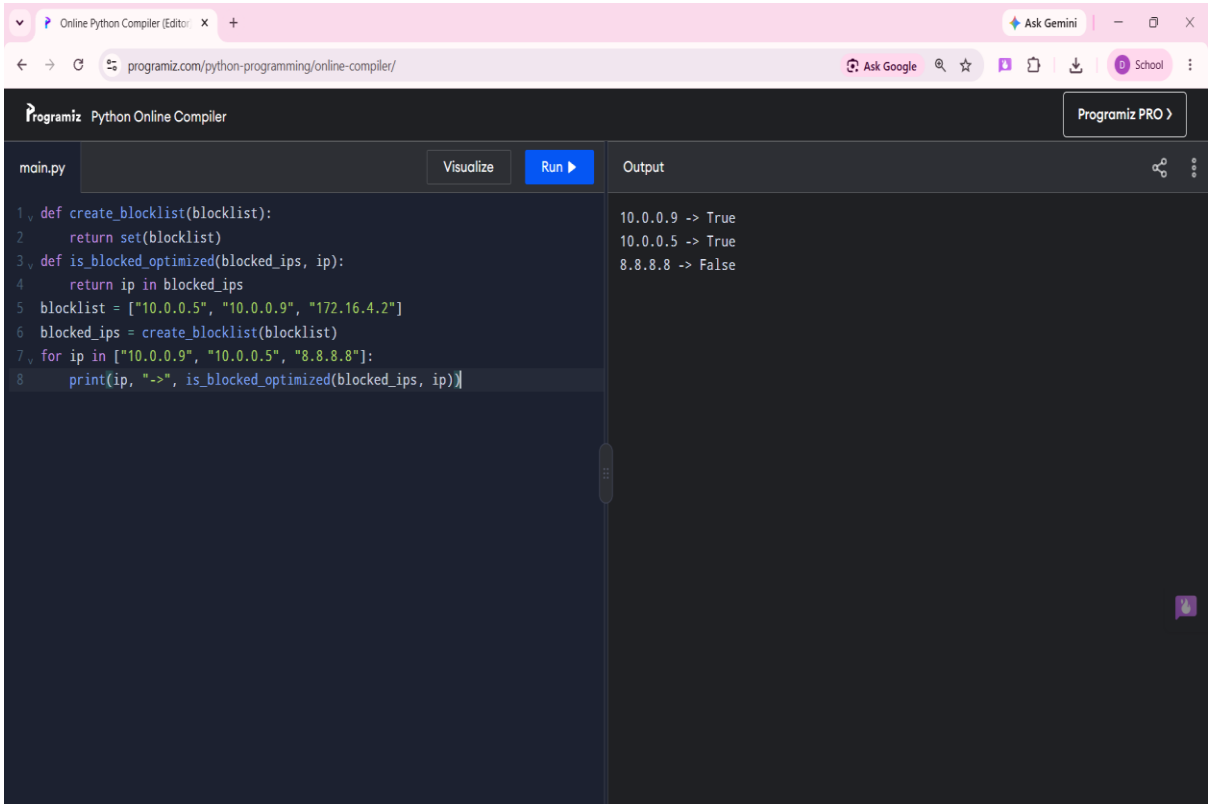


PART (c): OPTIMISE THE CORRECTED MODULE (10 MARKS)

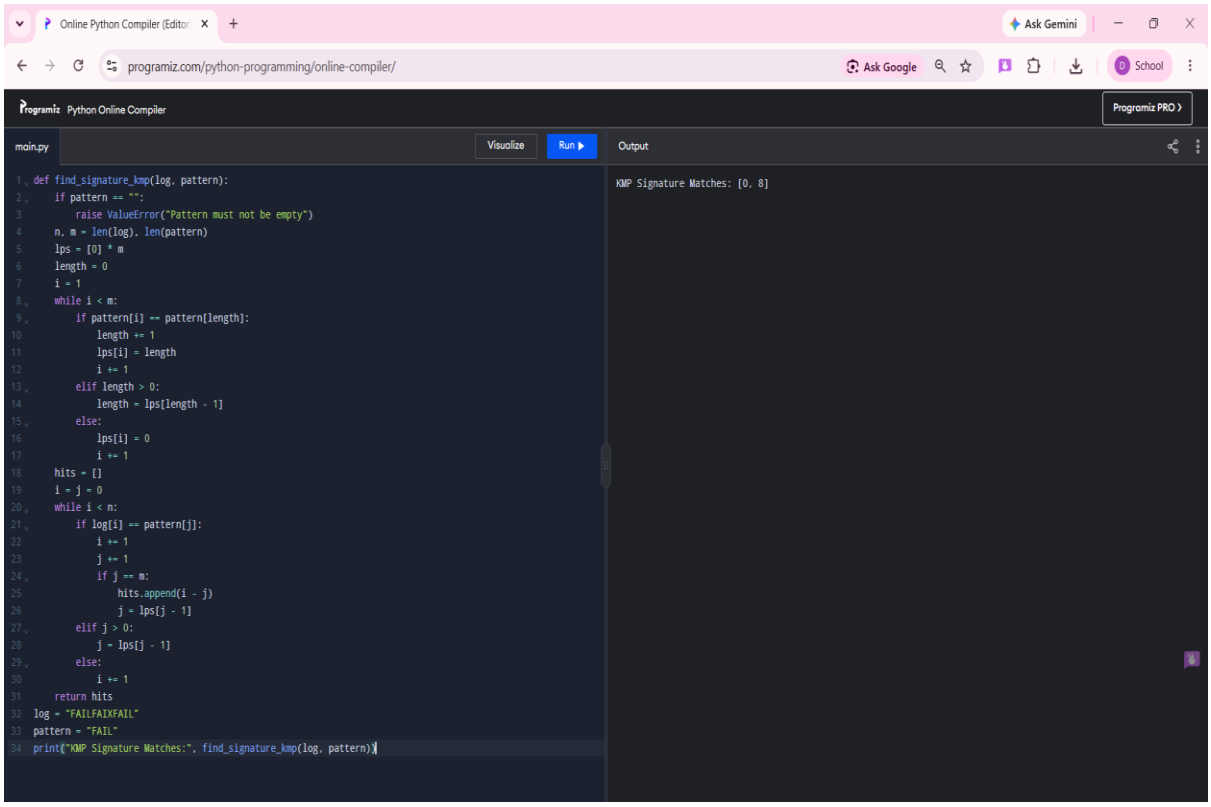
Optimising Alert Sorting



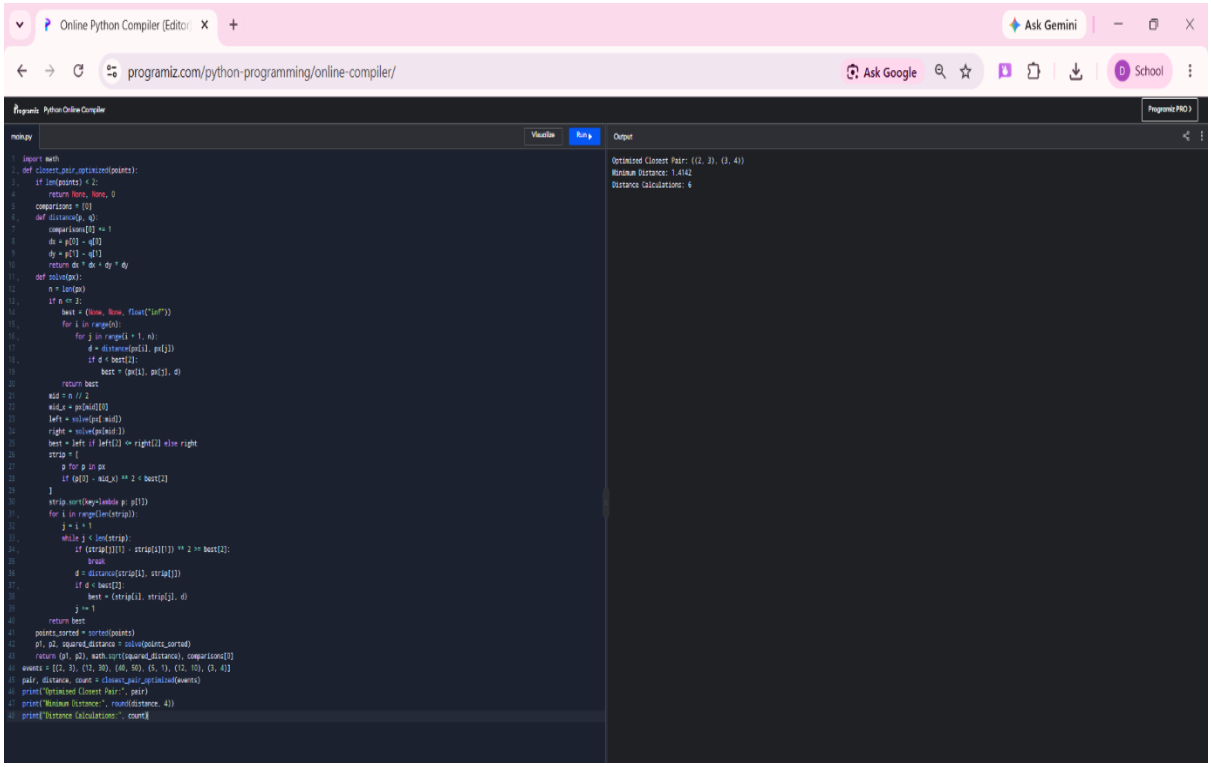
Optimising Blocklist Lookup



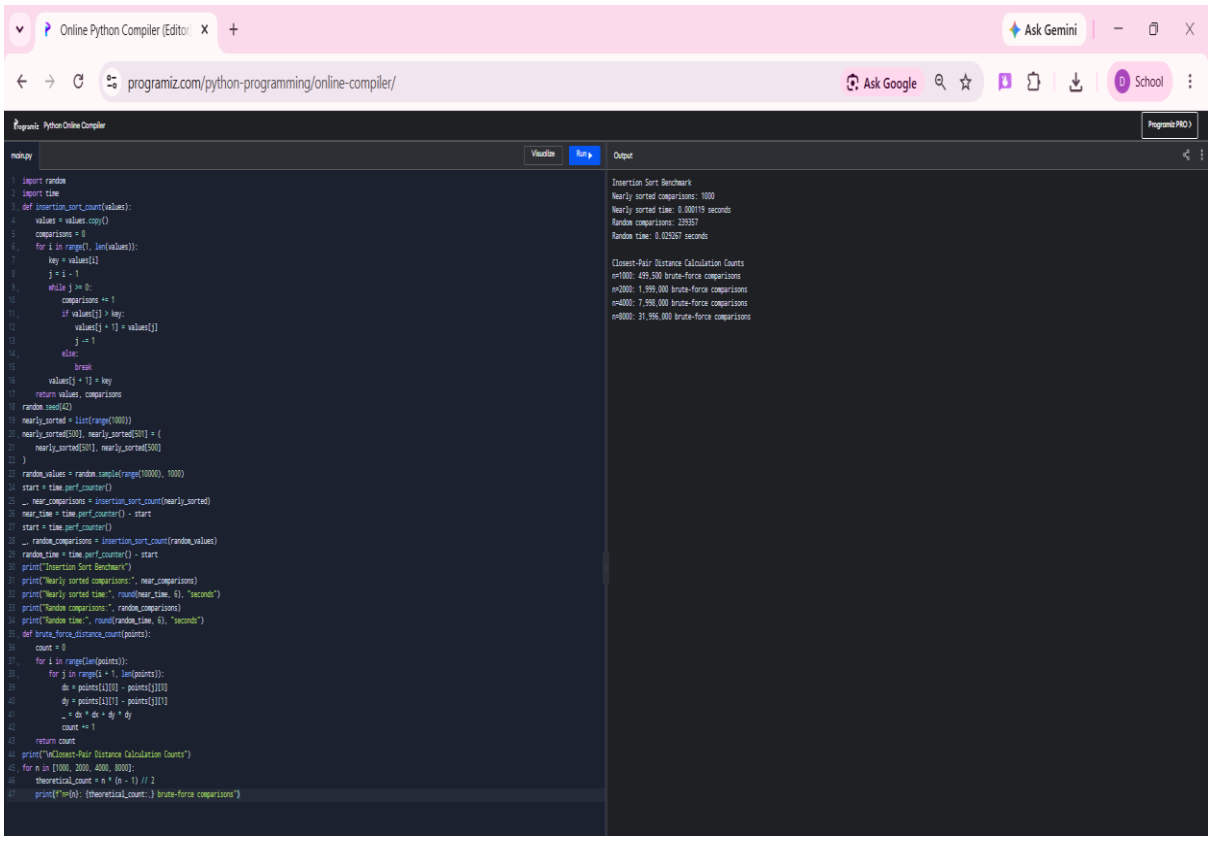
Optimising Signature Matching



Optimising Closest-Login Detection

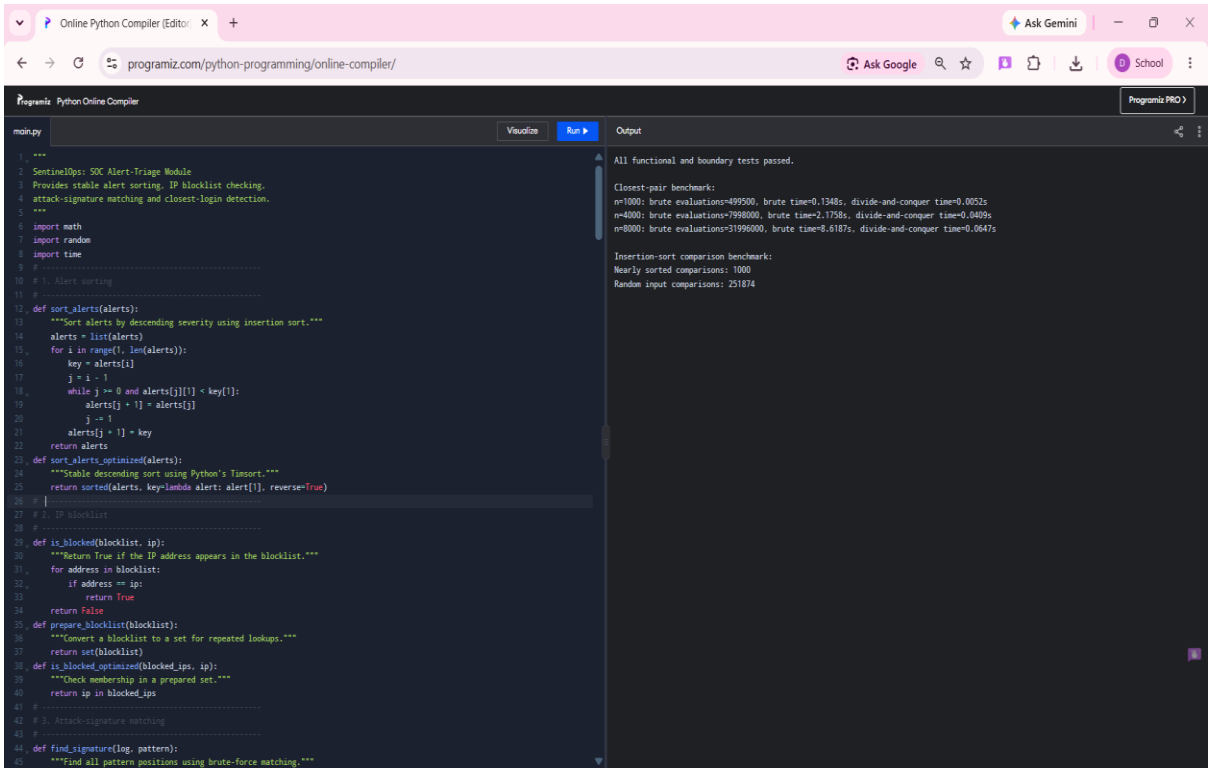


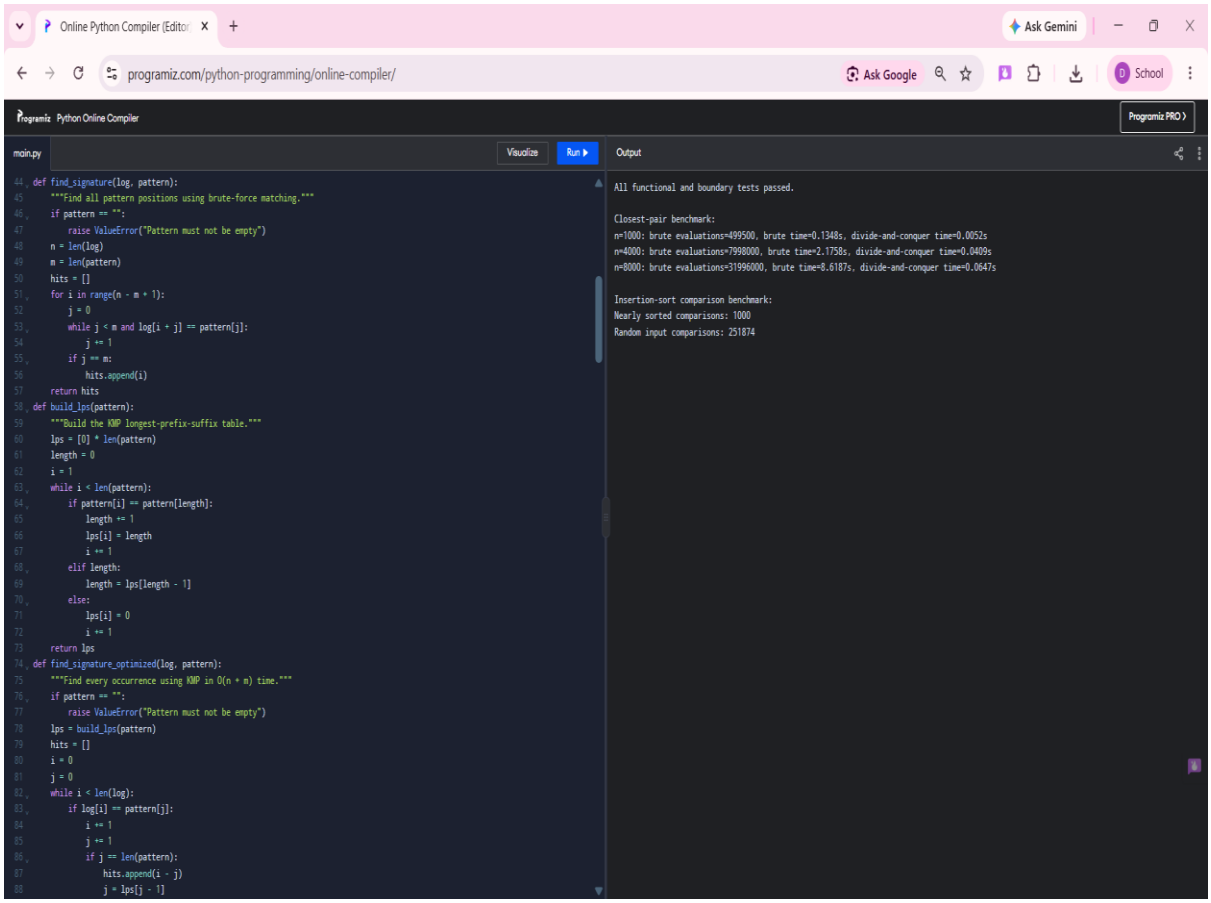
PART (d): ANALYSE BEFORE AND AFTER (10 MARKS)

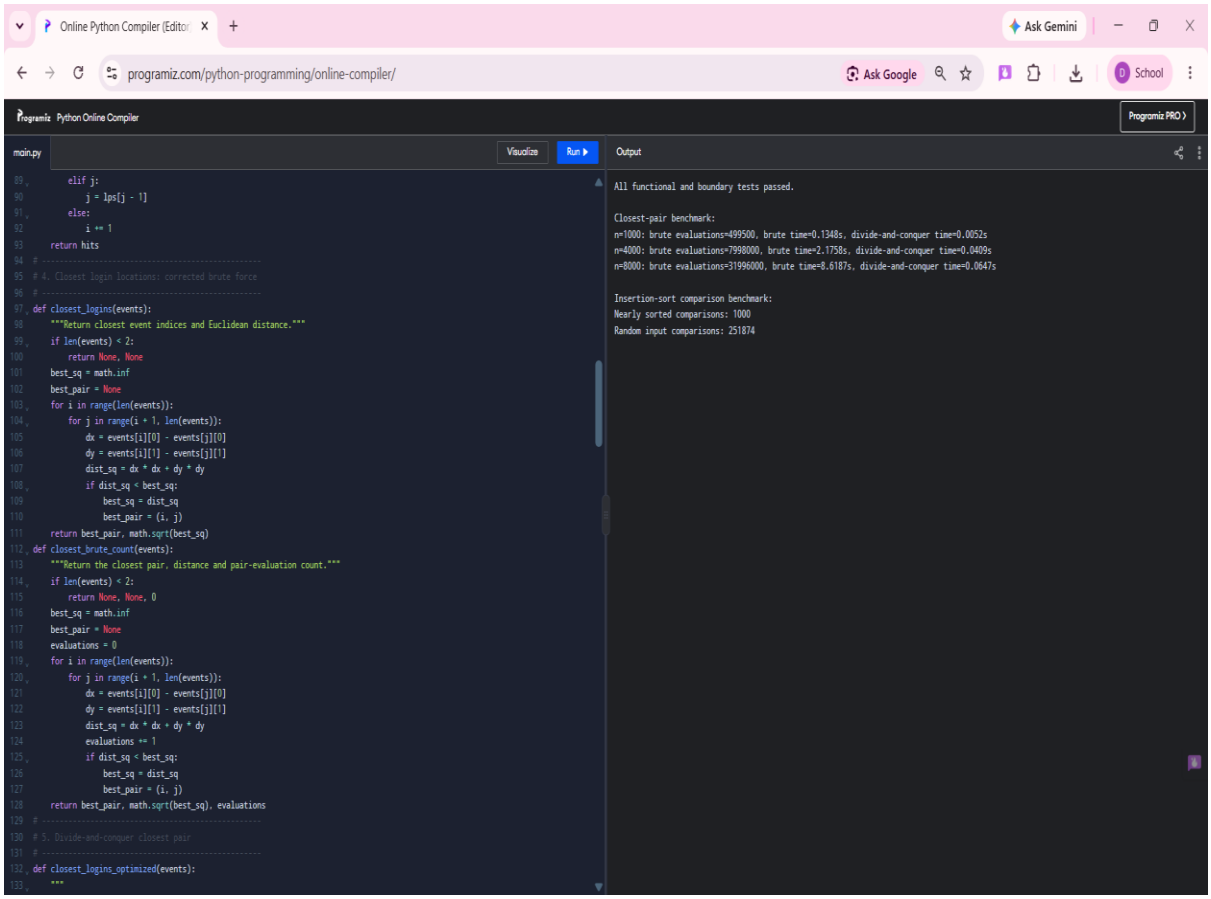


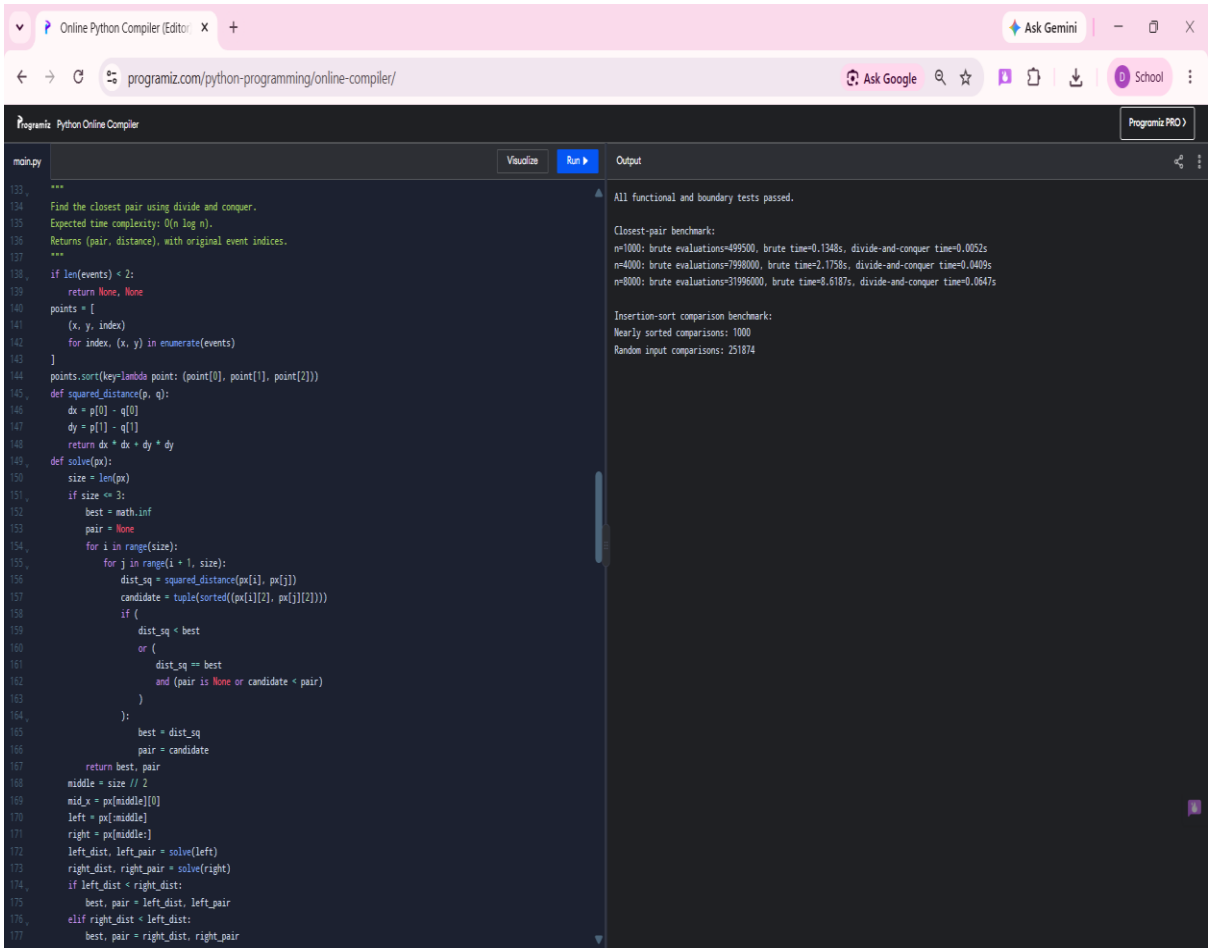
PART (e): DELIVER PRODUCTION-QUALITY CODE (8 MARKS)

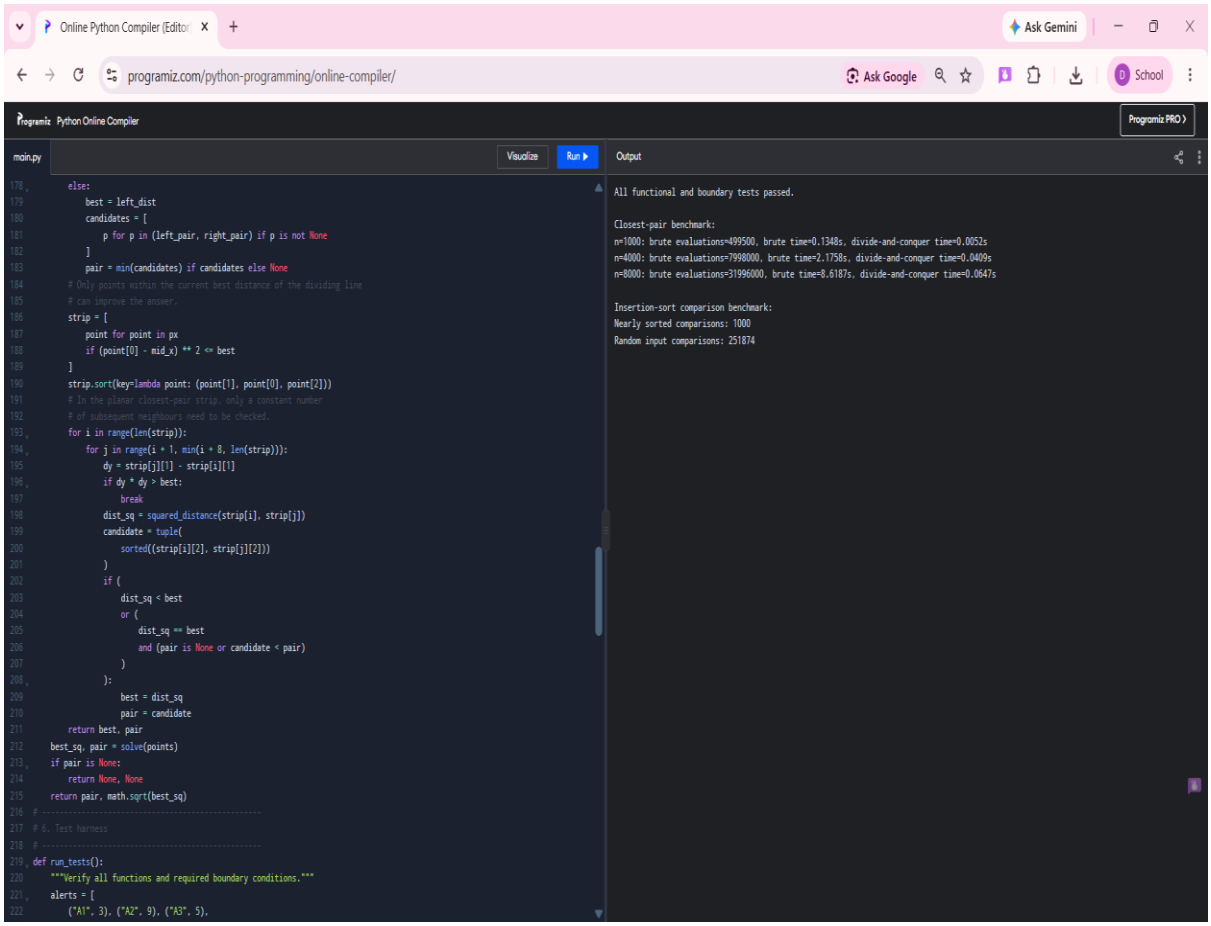
Complete Python Code

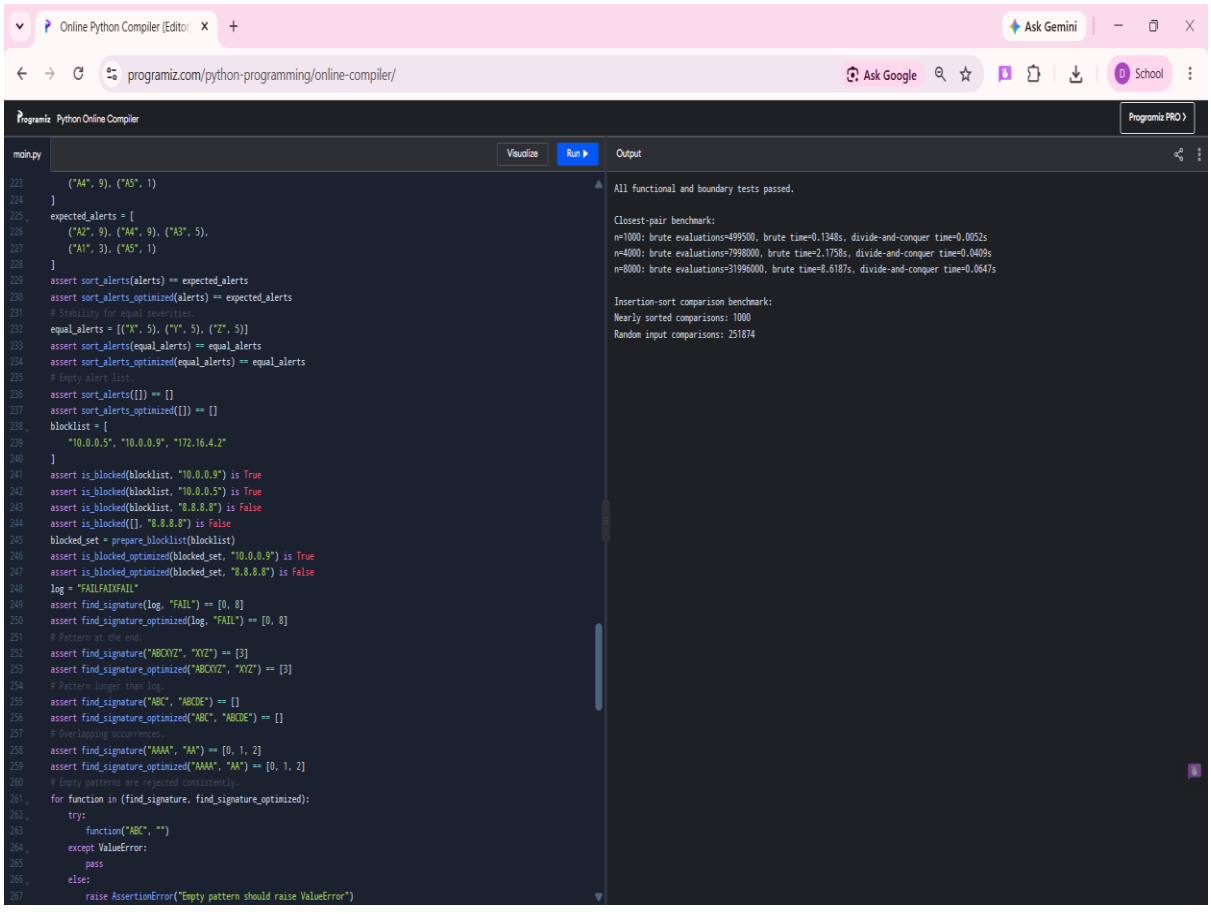


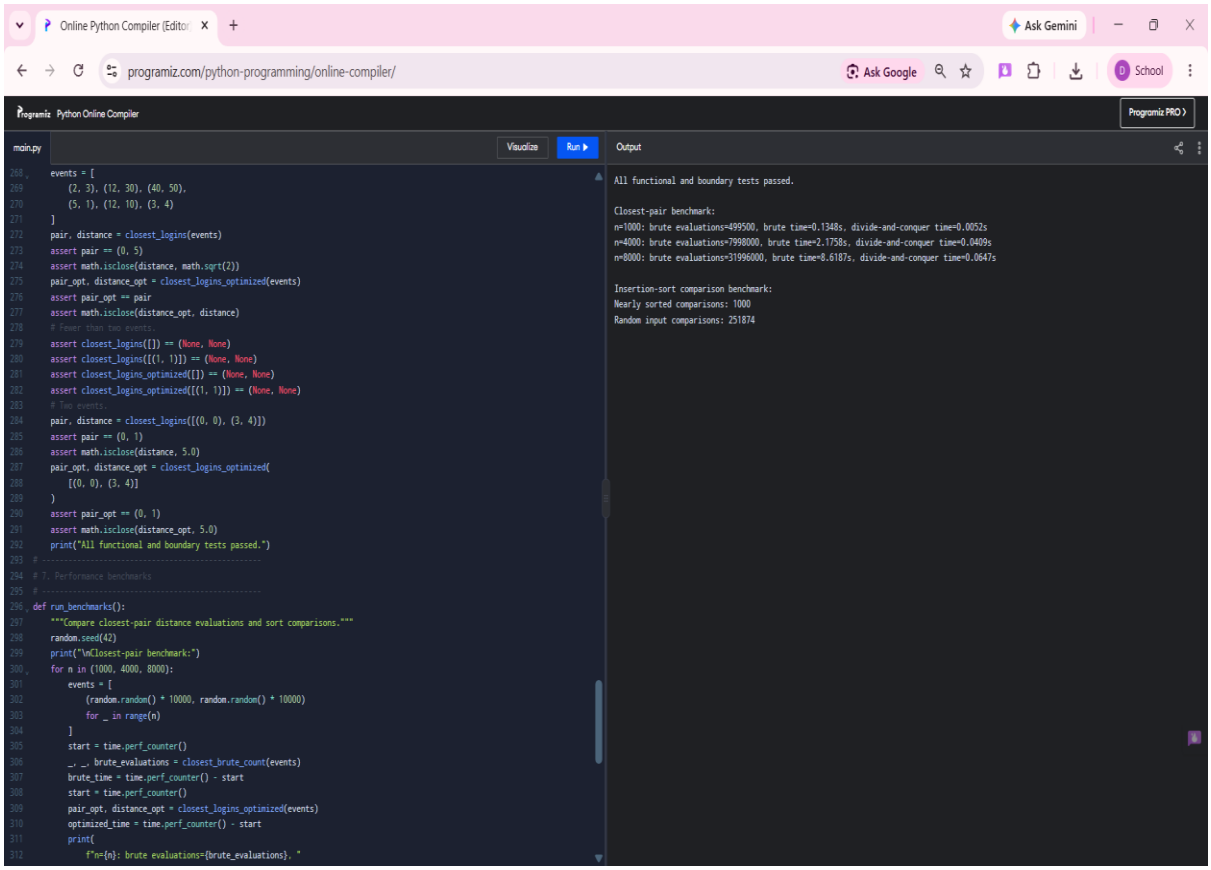


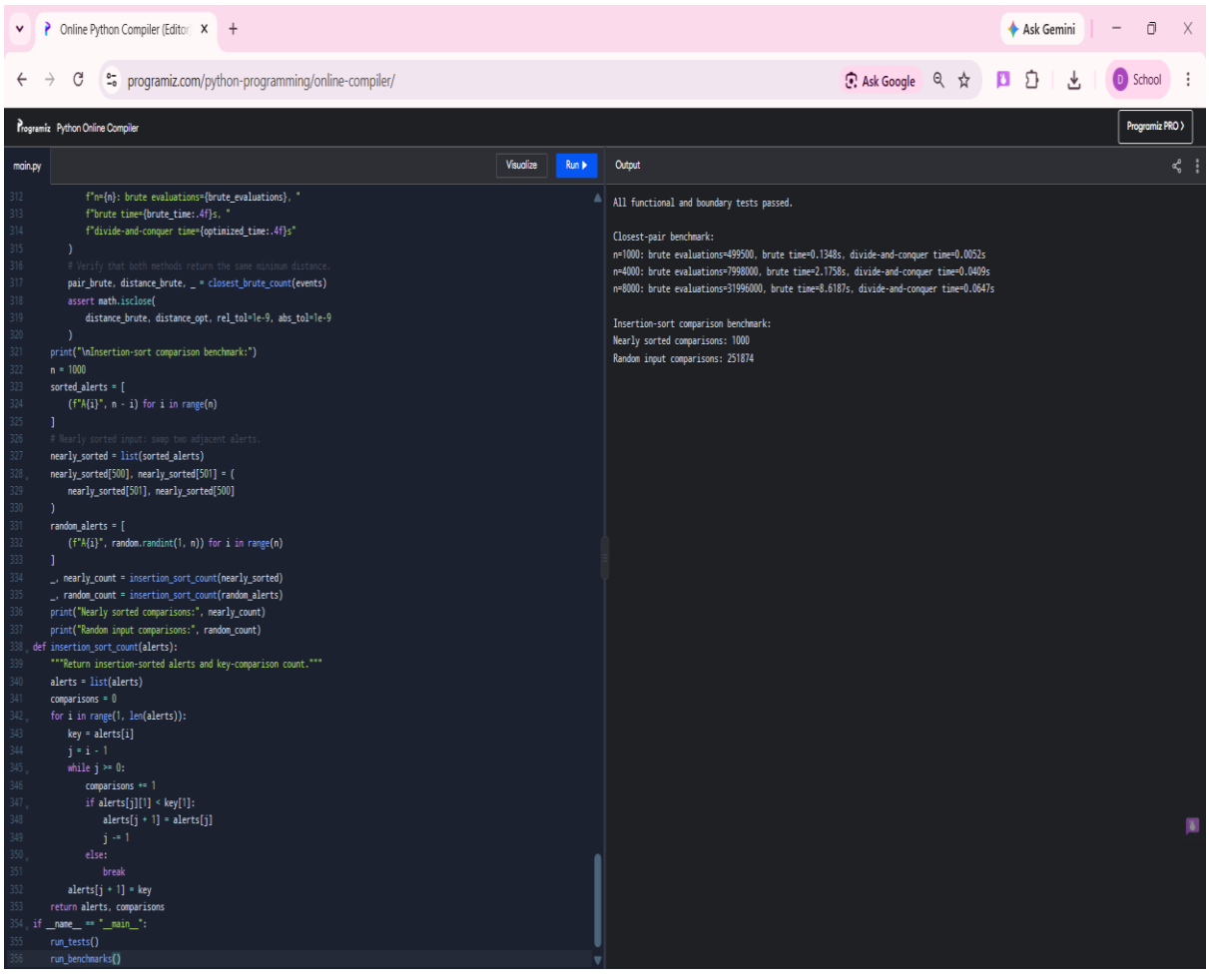

Online Python Compiler (Editor
Ask Gemini
programiz.com/python-programming/online-compiler/
Ask Google
School
Programiz Python Online Compiler
Programiz PRO
main.py
Visualize
Run
Output
elif j:
j = lps[j - 1]
else:
i += 1
return hits
----
4. Closest login locations: corrected brute force
----
def closest_logins(events):
"""Return closest event indices and Euclidean distance."""
if len(events) < 2:
return None, None
best_sq = math.inf
best_pair = None
for i in range(len(events)):
for j in range(i + 1, len(events)):
dx = events[i][0] - events[j][0]
dy = events[i][1] - events[j][1]
dist_sq = dx * dx + dy * dy
if dist_sq < best_sq:
best_sq = dist_sq
best_pair = (i, j)
return best_pair, math.sqrt(best_sq)
def closest_brute_count(events):
"""Return the closest pair, distance and pair-evaluation count."""
if len(events) < 2:
return None, None, 0
best_sq = math.inf
best_pair = None
evaluations = 0
for i in range(len(events)):
for j in range(i + 1, len(events)):
dx = events[i][0] - events[j][0]
dy = events[i][1] - events[j][1]
dist_sq = dx * dx + dy * dy
evaluations += 1
if dist_sq < best_sq:
best_sq = dist_sq
best_pair = (i, j)
return best_pair, math.sqrt(best_sq), evaluations
----
5. Divide-and-conquer closest pair
----
def closest_logins_optimized(events):
"""
All functional and boundary tests passed.
Closest-pair benchmark:
n=1000: brute evaluations=499500, brute time=0.1348s, divide-and-conquer time=0.0052s
n=4000: brute evaluations=7998000, brute time=2.1758s, divide-and-conquer time=0.0409s
n=8000: brute evaluations=31996000, brute time=8.6187s, divide-and-conquer time=0.0647s
Insertion-sort comparison benchmark:
Nearly sorted comparisons: 1000
Random input comparisons: 251874


Online Python Compiler (Editor
Ask Gemini
programiz.com/python-programming/online-compiler/
Ask Google
School
Programiz Python Online Compiler
Programiz PRO
main.py
Visualize
Run
Output
"""
Find the closest pair using divide and conquer.
Expected time complexity: O(n log n).
Returns (pair, distance), with original event indices.
"""
if len(events) < 2:
return None, None
points = [
(x, y, index)
for index, (x, y) in enumerate(events)
]
points.sort(key=lambda point: (point[0], point[1], point[2]))
def squared_distance(p, q):
dx = p[0] - q[0]
dy = p[1] - q[1]
return dx * dx + dy * dy
def solve(px):
size = len(px)
if size <= 3:
best = math.inf
pair = None
for i in range(size):
for j in range(i + 1, size):
dist_sq = squared_distance(px[i], px[j])
candidate = tuple(sorted((px[i][2], px[j][2])))
if (
dist_sq < best
or (
dist_sq == best
and (pair is None or candidate < pair)
)
):
best = dist_sq
pair = candidate
return best, pair
middle = size // 2
mid_x = px[middle][0]
left = px[:middle]
right = px[middle:]
left_dist, left_pair = solve(left)
right_dist, right_pair = solve(right)
if left_dist < right_dist:
best, pair = left_dist, left_pair
elif right_dist < left_dist:
best, pair = right_dist, right_pair
All functional and boundary tests passed.
Closest-pair benchmark:
n=1000: brute evaluations=499500, brute time=0.1348s, divide-and-conquer time=0.0052s
n=4000: brute evaluations=7998000, brute time=2.1758s, divide-and-conquer time=0.0409s
n=8000: brute evaluations=31996000, brute time=8.6187s, divide-and-conquer time=0.0647s
Insertion-sort comparison benchmark:
Nearly sorted comparisons: 1000
Random input comparisons: 251874


Online Python Compiler (Editor
Ask Gemini
programiz.com/python-programming/online-compiler/
Ask Google
School
Programiz Python Online Compiler
Programiz PRO
main.py
Visualize
Run
Output
All functional and boundary tests passed.
Closest-pair benchmark:
n=1000: brute evaluations=499500, brute time=0.1348s, divide-and-conquer time=0.0052s
n=4000: brute evaluations=7998000, brute time=2.1758s, divide-and-conquer time=0.0409s
n=8000: brute evaluations=31996000, brute time=8.6187s, divide-and-conquer time=0.0647s
Insertion-sort comparison benchmark:
Nearly sorted comparisons: 1000
Random input comparisons: 251874


Online Python Compiler (Editor
Ask Gemini
programiz.com/python-programming/online-compiler/
Ask Google
School
Programiz Python Online Compiler
Programiz PRO
main.py
Visualize
Run
Output
All functional and boundary tests passed.
Closest-pair benchmark:
n=1000: brute evaluations=499500, brute time=0.1348s, divide-and-conquer time=0.0052s
n=4000: brute evaluations=7998000, brute time=2.1758s, divide-and-conquer time=0.0409s
n=8000: brute evaluations=31996000, brute time=8.6187s, divide-and-conquer time=0.0647s
Insertion-sort comparison benchmark:
Nearly sorted comparisons: 1000
Random input comparisons: 251874


Online Python Compiler (Editor
Ask Gemini
programiz.com/python-programming/online-compiler/
Ask Google
School
Programiz Python Online Compiler
Programiz PRO
main.py
Visualize
Run
Output
events = [
(2, 3), (12, 30), (40, 50),
(5, 1), (12, 10), (3, 4)
]
pair, distance = closest_logins(events)
assert pair == (0, 5)
assert math.isclose(distance, math.sqrt(2))
pair_opt, distance_opt = closest_logins_optimized(events)
assert pair_opt == pair
assert math.isclose(distance_opt, distance)
Fewer than two events.
assert closest_logins([]) == (None, None)
assert closest_logins([(1, 1)]) == (None, None)
assert closest_logins_optimized([]) == (None, None)
assert closest_logins_optimized([(1, 1)]) == (None, None)
Two events.
pair, distance = closest_logins([(0, 0), (3, 4)])
assert pair == (0, 1)
assert math.isclose(distance, 5.0)
pair_opt, distance_opt = closest_logins_optimized(
[(0, 0), (3, 4)]
)
assert pair_opt == (0, 1)
assert math.isclose(distance_opt, 5.0)
print("All functional and boundary tests passed.")
--
7. Performance benchmarks
--
def run_benchmarks():
"""Compare closest-pair distance evaluations and sort comparisons."""
random.seed(42)
print("\nClosest-pair benchmark:")
for n in (1000, 4000, 8000):
events = [
(random.random() * 10000, random.random() * 10000)
for _ in range(n)
]
start = time.perf_counter()
_, _, brute_evaluations = closest_brute_count(events)
brute_time = time.perf_counter() - start
start = time.perf_counter()
pair_opt, distance_opt = closest_logins_optimized(events)
optimized_time = time.perf_counter() - start
print(
f"n={n}: brute evaluations={brute_evaluations}, "
All functional and boundary tests passed.
Closest-pair benchmark:
n=1000: brute evaluations=499500, brute time=0.1348s, divide-and-conquer time=0.0052s
n=4000: brute evaluations=7998000, brute time=2.1758s, divide-and-conquer time=0.0409s
n=8000: brute evaluations=31996000, brute time=8.6187s, divide-and-conquer time=0.0647s
Insertion-sort comparison benchmark:
Nearly sorted comparisons: 1000
Random input comparisons: 251874


Online Python Compiler (Editor
Ask Gemini
programiz.com/python-programming/online-compiler/
Ask Google
School
Programiz Python Online Compiler
Programiz PRO
main.py
Visualize
Run
Output
f"n={n}: brute evaluations={brute_evaluations}, "
f"brute time={brute_time:.4f}s, "
f"divide-and-conquer time={optimized_time:.4f}s"
)
Verify that both methods return the same minimum distance.
pair_brute, distance_brute, _ = closest_brute_count(events)
assert math.isclose(
distance_brute, distance_opt, rel_tol=1e-9, abs_tol=1e-9
)
print("\nInsertion-sort comparison benchmark:")
n = 1000
sorted_alerts = [
(f"A{i}", n - i) for i in range(n)
]
Nearly sorted input: swap two adjacent alerts.
nearly_sorted = list(sorted_alerts)
nearly_sorted[500], nearly_sorted[501] = (
nearly_sorted[501], nearly_sorted[500]
)
random_alerts = [
(f"A{i}", random.randint(1, n)) for i in range(n)
]
_, nearly_count = insertion_sort_count(nearly_sorted)
_, random_count = insertion_sort_count(random_alerts)
print("Nearly sorted comparisons:", nearly_count)
print("Random input comparisons:", random_count)
def insertion_sort_count(alerts):
"""Return insertion-sorted alerts and key-comparison count."""
alerts = list(alerts)
comparisons = 0
for i in range(1, len(alerts)):
key = alerts[i]
j = i - 1
while j >= 0:
comparisons += 1
if alerts[j][1] < key[1]:
alerts[j + 1] = alerts[j]
j -= 1
else:
break
alerts[j + 1] = key
return alerts, comparisons
if __name__ == "__main__":
run_tests()
run_benchmarks()
All functional and boundary tests passed.
Closest-pair benchmark:
n=1000: brute evaluations=499500, brute time=0.1348s, divide-and-conquer time=0.0052s
n=4000: brute evaluations=7998000, brute time=2.1758s, divide-and-conquer time=0.0409s
n=8000: brute evaluations=31996000, brute time=8.6187s, divide-and-conquer time=0.0647s
Insertion-sort comparison benchmark:
Nearly sorted comparisons: 1000
Random input comparisons: 251874